

11.4 Using Aider

After this introduction to the basic functions, we took a look at how Aider performs in real-life work. Our conclusion is a little like the basic tenor of this book: it depends on the complexity of the project, the task, the programming languages and libraries used, and so on.

11.4.1 Example 1: Sorting Photos

The following example shows you what using Aider can look like for a new project. You want to develop a Python script that moves photos to directories of type 2024/09 based on the date they were taken.

Prompt: I have a directory with photos (JPEGs with EXIF). I need a Python script to move the files to a new location according to their creation date, with year and month as subdirectories, for example, destination-directory/2024/08/myphoto.jpg. The source and destination directories are passed as arguments. If the destination directory is empty, create it.

Certainly! I'll create a Python script that accomplishes this task. Because you haven't shared any existing files, I'll create a new file called `organize_photos.py` with the required functionality. Here's the script.

[50 lines of code, more explanation]

To use this script, you'll need to install the Pillow library:

```
pip install Pillow
```

The script looks good at first glance. To easily set up a Python environment for testing, ask for a requirements file.

Prompt: Please create `requirements.txt`.

Now, open a second terminal window or a tab, and set up the test environment there. (You can, of course, also create the environment in a directory other than

the project directory if you prefer.)

```
python3 -m venv .
source bin/activate
pip3 install -r requirements.txt
python3 organize_photos.py ~/unsorted-pictures ~/pictures-by-date
```

Tip: Work in Small Steps

Even if Aider acts much more independently than GitHub Copilot, for example, there is a common denominator between current AI tools, regardless of their level. Prompts with too many or too complex tasks overwhelm Aider. Develop your project or solve your task in small steps. Perhaps it gives you some satisfaction that AI tools, for all their actual or ostensible “intelligence,” are still frighteningly limited in this respect.

The script works straight away. Great! Now you want to add some features to the script. First of all, you want an option so that the script doesn’t move the photos but copies them. This is very useful, especially for testing purposes.

Prompt: Add an option to copy instead of move the photos.

Certainly! I’ll modify the `organize_photos.py` script . . .

[code changes and explanation]

These changes will allow the user to specify whether they want to copy or move the photos. The script can now be run like this:

```
python organize_photos.py <source_directory> \
    <destination_directory> copy
```

or

```
python organize_photos.py <source_directory> \
    <destination_directory> move
```

Aider has implemented the option as an additional parameter that you should specify after the source and destination directories. That doesn’t make you happy because you were thinking more of a parameter like `--copy` as in Linux or macOS.

Aider isn't clairvoyant, but the tool isn't offended if it has to rebuild the code in a further step.

Prompt: Please use a Linux-style option (`--copy`) that can be passed at any position.

Aider automatically performs a syntax check after each change to detect errors. In this case, Aider added `sys.exit` to the code, but obviously forgot `import sys`. Aider asks whether it should try to rectify the error.

```
Running: /Users/kofler/no-sync/aider/bin/python3.12 \
-m flake8 --select=E9,F821,F823,F831,F406,\
F407,F701,F702,F704,F706 --show-source \
--isolated organize_photos.py

organize_photos.py:55:9: F821 undefined name 'sys'
sys.exit(1)

Attempt to fix lint errors? y
```

The error is trivial, and Aider manages to correct it. Then, you return to your test terminal and try out the new `--copy` option. Everything works fine. Now you ask for further improvement such as two additional options and the use of a separate directory for photos without EXIF data. (So far, the script has used *File Modification Time* to sort such photos, but this is rarely useful.)

Prompt: Add two more options: `--quiet` to suppress output (but show error messages) and `--help` to show a help text; also add short options `-c` for `--copy`, `-h` for `--help`, and `-q` for `--quiet`.

Prompt: Change the handling for files without EXIF data or with invalid EXIF data: move/copy these files into a special directory `no-exif-data`; also show a warning, even when `--quiet` is set.

You take some of the photos with your iPhone that uses the High Efficiency Image Coding (HEIC) format as standard. Wouldn't it be great if your script could cope with this type of image?

Prompt: Next, add support for HEIC files.

To add support for HEIC files, we need to make a few changes to the `organize_photos.py` file. We'll need to add the `pillow-heif` library to handle HEIC files, update the file extension check, and modify the `get_creation_date` function to work with HEIC files. Here are the necessary changes . . .

To test this in the second terminal, you must first repeat `pip3 install -r requirements.txt` to load the additionally required `pillow-heif` module into the virtual environment. After that, problems occur for the first time: the script indicates that it can't find any EXIF data in the HEIC files. You use an image editor to check this. The EXIF tags are definitely present. But `pillow-heif` apparently has problems reading them.

Now you've reached the moment when you have to deal with the code for the first time. (Keep in mind that you have *not* touched a *single line* of code yet! Up to this point in the example, you wouldn't have needed an editor at all.)

Your first idea is to check whether Aider is using current versions of `pillow` and `pillow-heif`. This isn't the case. You research on the internet, enter the latest version numbers in `requirements.txt`, repeat `pip3 install`, and test the program again. Unfortunately, there is no improvement.

Outdated Training Material

Old version numbers are a typical problem with common language models. The language model in this test from Anthropic was trained with old code and therefore often uses old versions. You should always take a look at version numbers in requirements files, Dockerfiles, or other text files that specify software versions! Update these numbers to the latest version to avoid possible security problems as well.

At this point, a new phase of program development begins: troubleshooting. You point out the problem to Aider and ask for it to be rectified.

Prompt: The script doesn't work for HEIC files (shows warning "no valid EXIF data . . ."); please try to fix it.

Aider builds a case distinction into the code that differentiates between JPEG and HEIF files. However, both code branches contain exactly the same code, so the faulty behavior doesn't change. Use `/undo` to revoke the last commit. The response to your succinct statement that the fix was ineffective is surprising. Aider inserts unsolicited logging code to give you a chance to get to the bottom of the error.

Prompt: /undo

Prompt: Your last fix didn't work.

I apologize for the confusion. Let's try a different approach to fix the issue with HEIC files. We'll modify the `get_creation_date` function to handle both JPEG and HEIC files more explicitly. Here's the proposed change . . .

Additionally, let's add some debug logging to help identify any issues . . .

You test the program again and take a look at the numerous issues. It turns out that `pillow-heif` only sees a fraction of the approximately 30 EXIF attributes. The `DateTimeOriginal` attribute (tag 36867) parsed by the script is missing, but at least `DateTime` (tag 306) is visible. This attribute specifies the date and time at which the image was last changed. Many pictures are never changed so that the information matches the date on which they were taken.

Thus, you ask Aider to parse `DateTimeOriginal` first, but to consider `DateTime` in a second step if necessary. (At this point, we want to state that the error in reading the EXIF data was caused by shortcomings in the Pillow library or incompatibilities in the HEIC format, not by Aider. If you had developed the code without AI tools, you would have stumbled at this point too.)

Prompt: Apparently, the script can only access a limited set of EXIF properties for HEIC files; first try to read 36867; if this fails, use tag 306 instead.

Prompt: This works; now remove the debugging code.

Up to this point, Anthropic has incurred costs of \$0.26, which is manageable. Including debugging, the time required was just under 90 minutes. The final script consists of 85 lines of code.

In our opinion, the remarkable thing about this example isn't so much the result (if you're interested in the code, take a look at the sample files in this book), but rather the way to get there. We could have written the script ourselves in an editor with ChatGPT or Copilot support. But instead, we really delegated all the coding and just took care of the testing. This is an entirely new approach. Of course, you don't have to work exclusively in this way: you still have full control over your code and can change or extend it at any time before asking Aider for help with a new task.

Aider offers another advantage compared to conventional program development: the entire process is wonderfully documented, first by a log of all prompts and outputs in the `.aider.chat.history.md` file in the project directory, and second, by an easy-to-read Git message sequence:

```
git log --pretty=format:"%s" --reverse

feat: Implement photo organization script
feat: add requirements.txt
feat: add option to copy or move photos
feat: Add Linux-style --copy option to organize_photos script
fix: import sys module
feat: Add --quiet and --help options to organize_photos.py
feat: Handle files without EXIF data or with invalid EXIF data
feat: Add support for HEIC files
fix: Move `register_heif_opener()` call after its import
fix: Handle HEIC files in get_creation_date function
feat: Add fallback to DateTime EXIF tag for HEIC files
refactor: remove debugging code
```

11.4.2 Example 2: Web Scraping

Before you think that Aider is the new miracle tool, we want to bring you back down to earth with the next example. Our goal is to program a script that extracts the top five bestsellers from [www.sap-press.com](https://www.sap-press.com/bestsellers) by means of web scraping, which actually isn't a particularly difficult task. First, we ask Aider to use `/web` to load the bestseller page and display the five titles. The only purpose of this is to check whether Aider can handle the website at all.

Prompt: `/web` <https://www.sap-press.com/bestsellers>

Prompt: List the top five titles of <https://www.sap-press.com/bestsellers>.

1. Materials Management with SAP S/4HANA: Business Processes and Configuration
2. Cloud Integration with SAP Integration Suite: The Comprehensive Guide
3. Developing AI Applications: An Introduction
4. SAP Build: No-Code Development, Centralized Access, and Process Automation
5. SAP Signavio: Business Process Transformation

As we can see, Aider does understand the structure of the website. Now we want Aider to automate this task using a Python script.

Prompt: Please write a Python script to extract the top five bestseller titles from <https://www.sap-press.com/bestsellers>.

Prompt: Add a requirement file to the project.

In VS Code, we take a look at `bestseller_scraper.py` and `requirements.txt`. Both files look good. We set up a virtual environment again, install the modules, and test the program. Unfortunately, the result is disappointing: the script only outputs a heading, while the actual list remains empty. So, we ask for improvements.

Prompt: `bestseller_scraper.py` only outputs an empty list; please try to fix the code.

I understand that the `bestseller_scraper.py` is outputting an empty list, and we need to fix the code. Let's analyze and modify the script to address this issue. The problem likely lies in how we're selecting elements from the web page. We'll adjust the HTML parsing to correctly identify and extract the book titles.

[new code]

Aider rebuilds the code a little and changes the names of the HTML classes the script searches for. To cut a long story short: Aider fails again, as the script still returns an empty list. This is disappointing insofar as Aider can read, parse, and understand HTML pages via the `/web` command. However, this functionality relates to the content of the page rather than its HTML structure. After a few more attempts, we abort this test without any success. Aider provides no advantage here compared to "honest," in-house program development.

API Errors

The HTML comprehension problems described here are unfortunately not the only annoyance we encountered during our Aider tests. It happened relatively often that our prompts were answered with error messages indicating that the AI backend we were using (Anthropic) was currently overloaded or didn't respond for some other reason. It usually helped to simply run the prompt again.

Of course, such API errors are most likely not the fault of Aider, but have to do with shortcomings of the respective AI provider. However, we've never had as

many issues with any other tool presented in this book as with Aider. That's odd!

11.4.3 Example 3: Implementing Changes in a Large Project

Most recently, we tried to use Aider to make changes to a complex, real-world project consisting of several hundred files (PHP, JavaScript, Cascading Style Sheets [CSS]). The following paragraphs show a few exemplary prompts. Our goal was to change details of an HTML table that gets output by functions of the `statistics.php` file. (All functions mentioned in the prompts can be found in this file.) As the data displayed in the HTML table originates from a database, we've also adopted the database schema in the context.

Prompt: `/add accounts/statistics.php`

Prompt: `/read doc/accounting-schema.sql`

Prompt: In `getUsageStatistics()`, add exception handling for the call `connectToCustomerDatabase()`; if the connection fails, continue loop with the next customer.

Prompt: `showUsage()` displays the customer name in the first column of an HTML table; make this name a link to `editCustomers.php?id=nnn` where `nnn` is the customer ID.

Prompt: Still in the first column of the HTML table created by `showUsage()`: after the link, add the year of the initial order; you get the necessary data from the database column `customers.initialOrderDate`.

Our findings were mixed, not totally bad, but not great either:

- We got the impression that only prompts that are very carefully and precisely formulated lead to the goal. Sometimes we wondered what takes longer: the Aider prompt input or a direct modification of the file with the support of a wizard integrated into the editor.
- Even with precisely formulated prompts, Aider didn't always do exactly what we expected. Sometimes, we revoked the entire commit using `/undo` and then tried again with a better prompt. At other times, we simply carried out the corrections ourselves, which often consisted of simple mix-ups of column or variable names. Of course, it's remarkable when Aider understands an instruction 90% of the time and carries it out correctly. But that still doesn't suffice for really efficient work.

Long story short is that purely instruction-oriented programming takes some getting used to, but is certainly exciting. However, we didn't have the feeling that Aider would save us much time. The benefits of this approach are probably greatest when you develop new code from scratch—be it a completely new project or a new method or class in an existing project.

11.4.4 More Prompting Examples and Video Tips

By default, Aider saves the prompt history, including all responses in the `.aider.chat.history.md` Markdown file. A collection of mostly short examples derived from this (each prompt plus response in visually appealing formatting) can be found at <https://aider.chat/examples/README.html>.

There is also a whole series of videos available on the internet showing Aider at work. Aider developers have compiled links to such videos at <https://aider.chat/docs/usage/tutorials.html>.

These examples show various aspects of the use of Aider and provide many application ideas. However, we had the impression that some examples were really optimized for Aider and therefore conveyed an overly optimistic picture. As with all other AI tools presented in this book, you must also expect problems and errors when using Aider in real life.